

- + close():evento emesso alla scadenza del timer. Il padre reagisce smontando il componente.

5.8.2.5) FileDropZone

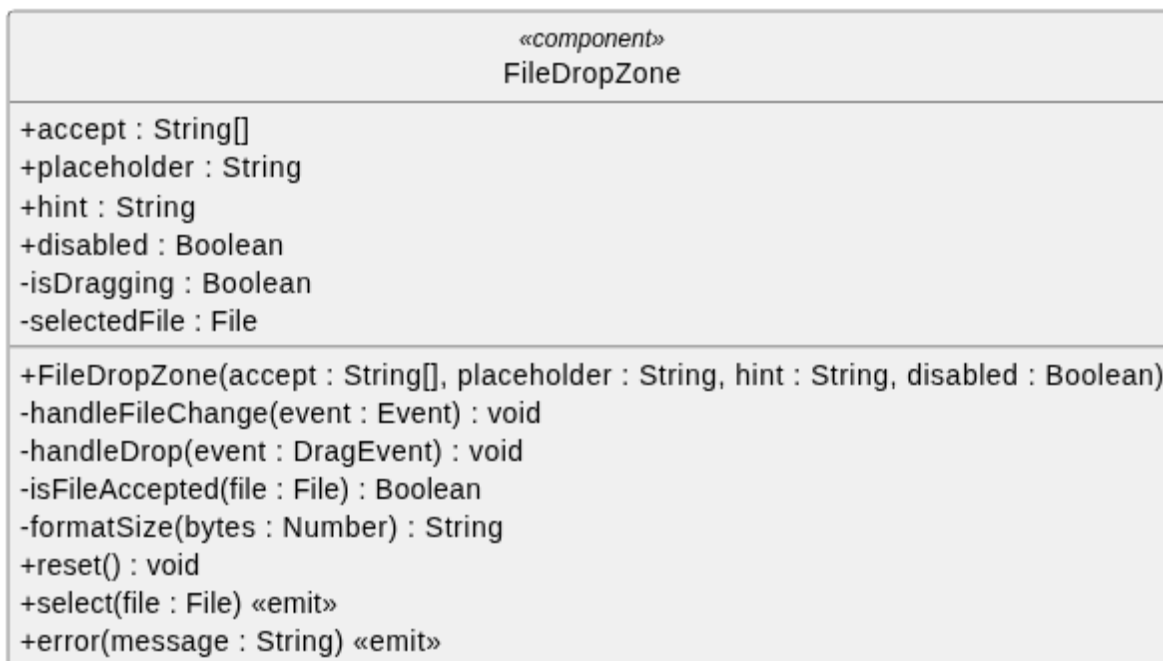


Figura 167: Componente - FileDropZone

Descrizione:

Componente generico che realizza un'area di caricamento file con supporto per drag & drop e selezione tramite click. Gestisce il feedback visivo durante il trascinamento, valida il tipo di file in base alle estensioni accettate e comunica il file selezionato al padre tramite evento. Non conosce cosa verrà fatto del file — l'upload o l'elaborazione sono responsabilità del widget che lo utilizza.

Attributi:

- - accept:String[]: Elenco di estensioni supportate.
- - placeholder: String: Testo del placeholder.
- - hint: String: Suggerimento testuale sui formati accettati.
- - disabled: Boolean: Flag per disabilitare l'interazione.
- - isDragging: Boolean: Indica se un file è attualmente trascinato sopra l'area di drop. Utilizzato per il feedback visivo.
- - selectedFile: File: Riferimento al file selezionato dall'utente, null se nessun file è stato scelto.

Metodi:

- + FileDropZone(accept: String[], placeholder: String, hint: String, disabled: Boolean):Costruttore. Riceve le estensioni accettate, il testo placeholder, un suggerimento sui formati supportati e un flag per disabilitare l'interazione.

- - `handleFileChange(event:Event)`: Metodo privato invocato alla selezione di un file tramite il file picker nativo del browser.
- - `handleDropEvent(event:DragEvent)`: Metodo privato invocato al rilascio di un file trascinato sull'area di drop. Valida il file e lo accetta o emette un errore.
- - `isFileAccepted(file:File)`: Metodo privato che verifica se l'estensione del file è tra quelle accettate.
- - `formatSize(bytes:Number)`: Metodo privato che converte una dimensione in byte in formato leggibile (Bytes, KB, MB).
- + `reset()`: Metodo pubblico esposto al padre tramite template ref. Resetta lo stato del componente rimuovendo il file selezionato.
- + `select(file:File)`: Evento emesso quando un file valido viene selezionato o trascinato.
- + `error(message:String)`: Evento emesso quando il file trascinato non supera la validazione del formato.

5.8.2.6) Device Form

«Component» DeviceForm
+fields: Map<String, String Number Boolean> +errors: Map<String, String null> -fieldComponents: FormField[]
+DeviceForm(fields: Map<String, String Number Boolean>, errors: Map<String, String null>)

Figura 168: Component - DeviceForm

Descrizione:

Componente di presentazione specifico per il dominio dei dispositivi. Si occupa esclusivamente di renderizzare l'interfaccia utente del modulo e di stabilire un binding bidirezionale con i dati forniti dal componente genitore. È agnostico rispetto al contesto: non sa se sta creando un nuovo dispositivo o modificandone uno esistente, e non esegue alcuna logica di validazione o salvataggio.

Attributi:

- + `fields`: Object: Oggetto reattivo iniettato dal padre contenente i valori dei campi del dispositivo.
- + `errors`: Object: Oggetto iniettato dal padre contenente gli eventuali messaggi di errore da visualizzare per ciascun campo.

Metodi:

Il componente è puramente dichiarativo e non espone metodi operativi. Espone unicamente il costruttore:

- + `DeviceForm(fields: Map<String, String | Number | Boolean>, errors: Map<String, String | null>)`

5.8.2.7) Asset Form

«Component» AssetForm
+fields: Map<String, String Number Boolean> +errors: Map<String, String null> -fieldComponents: FormField[]
+AssetForm(fields: Map<String, String Number Boolean>, errors: Map<String, String null>)

Figura 169: Component - AssetForm

Descrizione: Componente di presentazione (Dumb Component) delegato al rendering del modulo per la gestione degli Asset. Si occupa esclusivamente di renderizzare l'interfaccia utente del modulo e di stabilire un binding bidirezionale con i dati forniti dal componente genitore. È agnostico rispetto al contesto: non sa se sta creando un nuovo asset o modificandone uno esistente, e non esegue alcuna logica di validazione o salvataggio.

Attributi:

- + fields: Object: Oggetto reattivo iniettato dal padre contenente i valori dei campi del dispositivo.
- + errors: Object: Oggetto iniettato dal padre contenente gli eventuali messaggi di errore da visualizzare per ciascun campo.

Metodi:

Il componente è puramente dichiarativo e non espone metodi operativi. Espone unicamente il costruttore:

- + AssetForm(fields: Map<String, String | Number | Boolean>, errors: Map<String, String | null>)

5.8.3) Widget

5.8.3.1) AssetCreateWidget

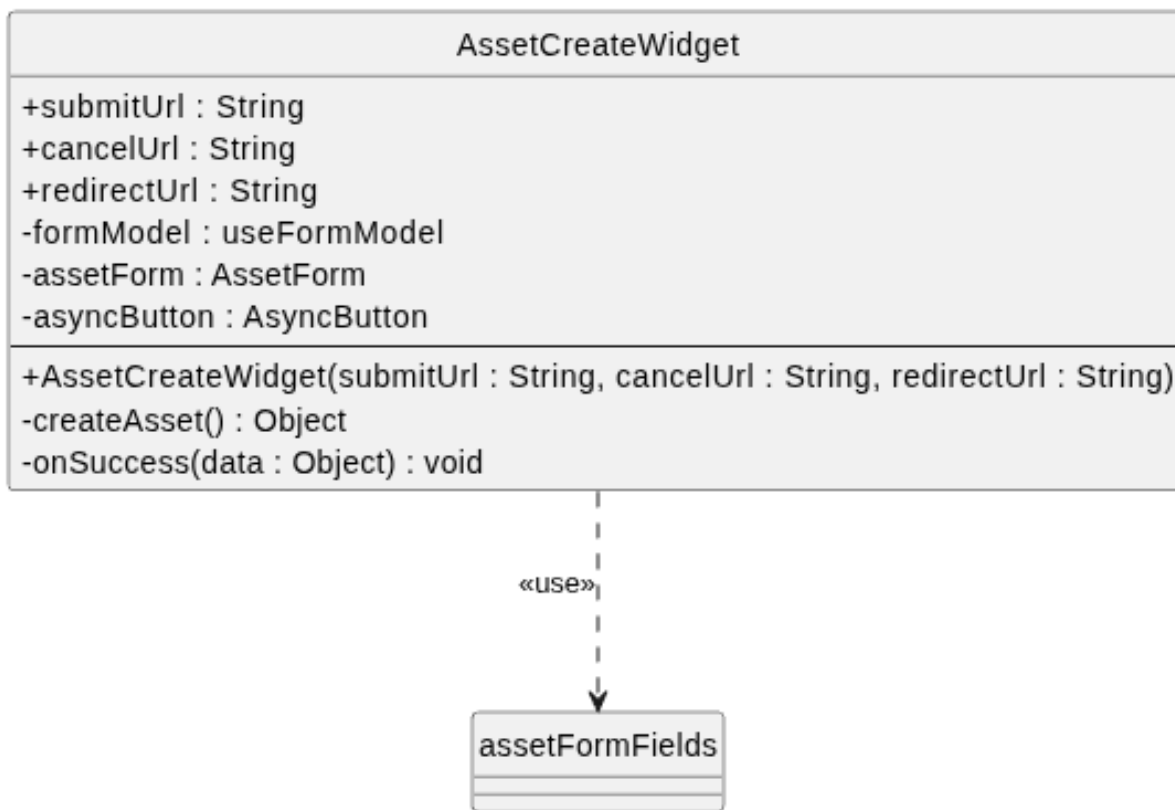


Figura 170: Widget - AssetCreateWidget

Descrizione: Rappresenta un'isola applicativa responsabile di orchestrare la creazione di un nuovo Asset. Agisce come controller di facciata: funge da ponte tra il livello di presentazione e il livello di comunicazione col backend. Detiene la logica di business specifica per questa casistica, ma delega il rendering grafico e la gestione reattiva ai componenti e ai composabile sottostanti.

Attributi:

- `+ submitUrl: String`: L'endpoint Flask a cui inviare il payload POST.
- `+ cancelUrl: String`: L'URL di fallback in caso di annullamento dell'operazione.
- `+ redirectUrl: String`: L'URL a cui reindirizzare l'utente in caso di successo.
- `- formModel: UseFormModel : Model` di riferimento per il componente.
- `- assetForm:AssetForm`: Componente che mostra graficamente il form degli asset.
- `- confirmButton:AsyncButton`: Componente che mostra graficamente il pulsante di conferma.

Metodi:

- `AssetCreateWidget(submitUrl : String, cancelUrl : String, redirectUrl : String)`: Costruttore che riceve esternamente gli url a cui corrispondono le azioni.

- - `createAsset(): Promise<Object>` : Metodo asincrono invocato dal bottone di salvataggio. Esegue la validazione invocando il composabile, compone il payload JSON e gestisce la chiamata di rete, catturando e smistando eventuali errori server-side, ritorna un oggetto json contenente la risposta.
- - `onSuccess(data: Object): void` : Callback eseguita al completamento positivo della chiamata.

5.8.3.2) AssetUpdateWidget

5.8.3.3) AssetDeleteWidget

5.9) Architettura Frontend: Modulo Decision Tree

Il frontend per la valutazione dei requisiti è progettato per essere reattivo e disaccoppiato. Per evitare di sovraccaricare il server a ogni interazione, una porzione della logica di dominio viene eseguita direttamente lato client, orchestrata da uno State Management centralizzato.

L'architettura si articola su quattro pilastri:

1. **Core di Valutazione (EvaluationEngine)**: Il client non si limita a mostrare dati passivi, ma ricalcola istantaneamente il percorso attivo dell'albero in base alle risposte dell'utente, offrendo un feedback visivo immediato tramite una struttura a oggetti polimorfa.
2. **Gestione dello Stato (DecisionTreeStore)**: Basato su Pinia, funge da «Single Source of Truth». Mantiene in memoria le risposte, lo stato dell'interfaccia, il nodo selezionato e delega i calcoli complessi all'EvaluationEngine.
3. **Interfaccia Utente (UI)**: Suddivisa in due macro-aree cooperanti. Il **Tree Canvas** disegna la topologia dell'albero delegando i calcoli geometrici al D3LayoutEngine, mentre la **Tree Sidebar** funge da pannello interattivo per leggere le domande e inserire le risposte, comunicando esclusivamente con lo Store.
4. **Integrazione Backend (EvaluationApiClient)**: Astrattizza la comunicazione HTTP con il server, assicurando che lo stato locale dell'applicazione sia costantemente sincronizzato con il database remoto in modo asincrono.

Nei paragrafi successivi verranno analizzati nel dettaglio i diagrammi delle classi dei singoli sottosistemi.

5.9.1) Architettura Interattiva della Sidebar

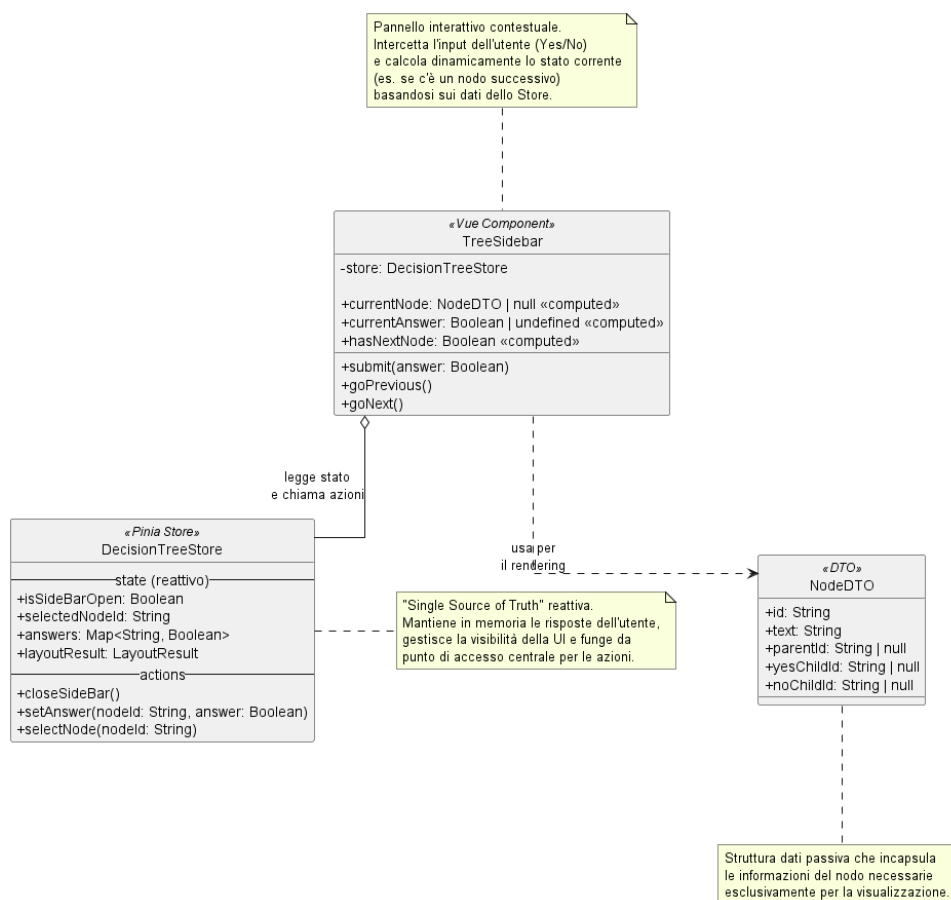


Figura 171: Architettura del componente TreeSidebar e del suo Store

Descrizione

Il diagramma illustra la relazione e la separazione delle responsabilità tra l'interfaccia utente laterale (*TreeSidebar*), il gestore dello stato globale (*DecisionTreeStore*) e l'oggetto di trasferimento dati (*NodeDTO*). Il componente visivo si interfaccia esclusivamente con lo Store per leggere lo stato reattivo e invocare le azioni di aggiornamento, utilizzando il DTO in modo puramente passivo per estrarre le informazioni testuali e topologiche necessarie al rendering a schermo.

Di seguito il dettaglio dei singoli componenti coinvolti nel flusso.

5.9.1.1) DecisionTreeStore

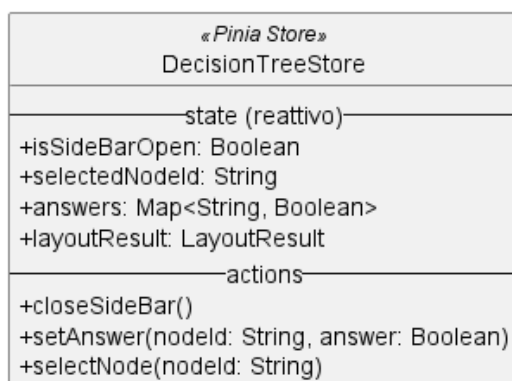


Figura 172: DecisionTreeStore

Descrizione

DecisionTreeStore è il modulo basato su Pinia che funge da «Single Source of Truth» (singola fonte di verità) per il frontend. Mantiene in memoria in modo reattivo le risposte dell'utente, gestisce la visibilità della UI e accentra il punto di accesso per tutte le modifiche di stato.

Stato Reattivo (State)

- + *isSideBarOpen*: Boolean — flag che determina se il pannello laterale è attualmente aperto o nascosto.
- + *selectedNodeId*: String — l'identificativo del nodo correntemente selezionato o cliccato dall'utente.
- + *answers*: Map<String, Boolean> — dizionario che associa l'ID di ogni nodo decisionale alla rispettiva risposta (Yes/No) fornita dall'utente.
- + *layoutResult*: LayoutResult — struttura dati che incapsula i risultati del calcolo geometrico (nodi e archi) necessari a renderizzare l'albero.

Metodi (Actions)

- + *closeSideBar()* — azione per chiudere il pannello laterale.
- + *setAnswer(nodeId: String, answer: Boolean)* — registra o aggiorna la risposta dell'utente per un dato nodo decisionale.
- + *selectNode(nodeId: String)* — azione per impostare un nodo come «selezionato», scatenando gli aggiornamenti reattivi sulla UI.

5.9.1.2) TreeSidebar

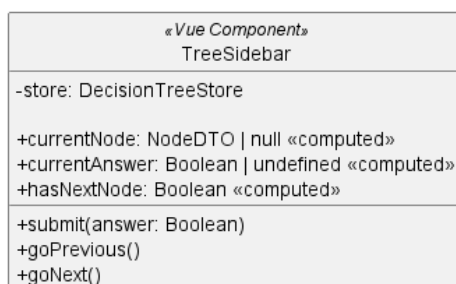


Figura 173: TreeSidebar

Descrizione

TreeSidebar è il componente Vue.js che implementa il pannello laterale interattivo. Il suo compito è intercettare l'input dell'utente e presentare le informazioni del nodo selezionato calcolando dinamicamente la navigazione basandosi sullo Store centrale.

Proprietà Compute (Computed)

- + `currentNode: NodeDTO | null` — recupera dinamicamente le informazioni del nodo selezionato in quel momento.
- + `currentAnswer: Boolean | undefined` — estrae dallo Store la risposta precedentemente data (se presente) per il nodo corrente.
- + `hasNextNode: Boolean` — calcola dinamicamente se esiste un nodo successivo navigabile partendo dalla situazione corrente.

Metodi

- + `submit(answer: Boolean)` — gestisce l'invio della risposta per la domanda a schermo, delegandone il salvataggio all'azione dello Store.
- + `goPrevious()` — innesca la navigazione per tornare al nodo precedente lungo il percorso di valutazione.
- + `goNext()` — innesca la navigazione per avanzare al nodo logico successivo all'interno del flusso.

5.9.1.3) NodeDTO

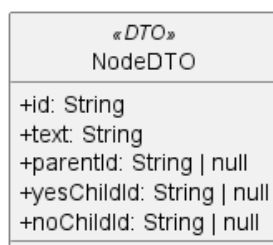


Figura 174: NodeDTO

Descrizione

NodeDTO (Data Transfer Object) è una struttura dati passiva utilizzata esclusivamente per il trasferimento e la formattazione dei dati. Incapsula le informazioni «grezze» del nodo in modo che il componente visivo possa stamparle a schermo senza dover accedere alla complessa logica di dominio.

Attributi

- + id: String — identificativo univoco del nodo.
- + text: String — il contenuto testuale della domanda o dell’esito.
- + parentId: String | null — riferimento all’ID del nodo padre, necessario per calcolare la navigazione a ritroso.
- + yesChildId: String | null — riferimento all’ID del nodo figlio in caso di risposta affermativa.
- + noChildId: String | null — riferimento all’ID del nodo figlio in caso di risposta negativa.

Metodi

NodeDTO non espone metodi, trattandosi di un oggetto dedicato al solo trasporto dati.

5.9.2) Architettura Visiva del Canvas (Tree Canvas)

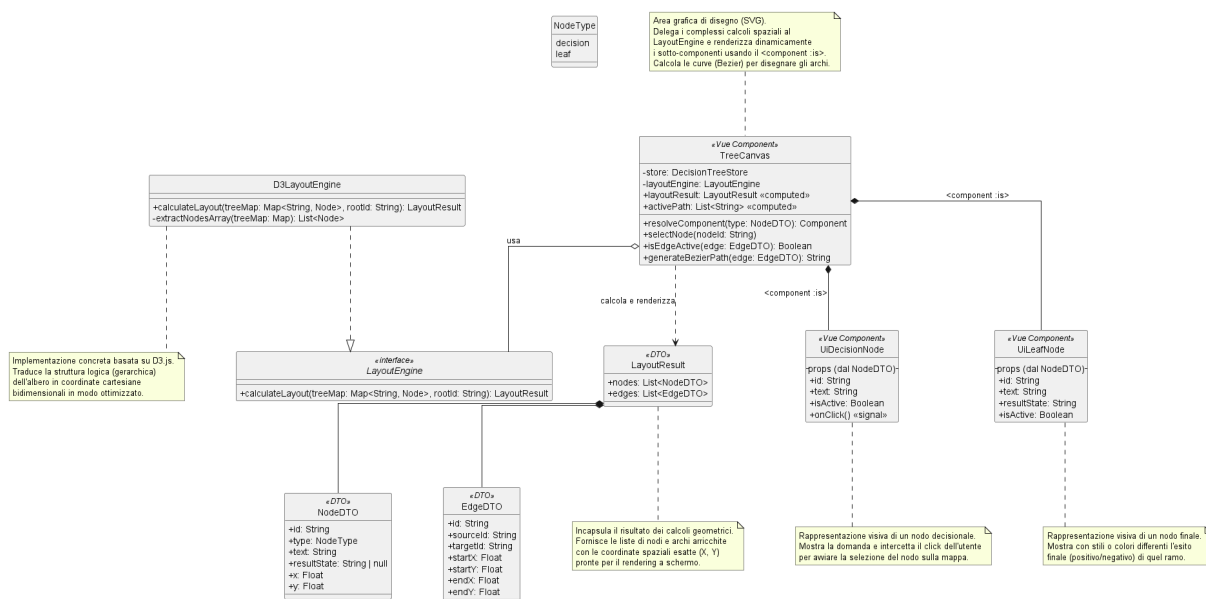


Figura 175: Architettura del componente TreeCanvas e del Layout Engine

Descrizione

Il diagramma illustra l'architettura dedicata alla renderizzazione topologica dell'albero decisionale. Per garantire prestazioni ottimali e codice pulito, il sistema separa nettamente il calcolo matematico delle coordinate visive (delegato a *D3LayoutEngine*) dal rendering effettivo a schermo (gestito dal componente *TreeCanvas* e dai suoi sotto-componenti *UiDecisionNode* e *UiLeafNode*).

Di seguito vengono analizzati nel dettaglio i componenti e le strutture dati che partecipano a questo flusso.

5.9.2.1) LayoutResult e i DTO Geometrici

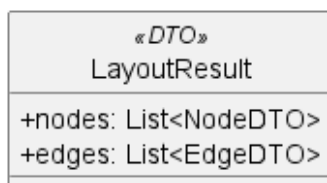


Figura 176: LayoutResult, NodeDTO ed EdgeDTO

Descrizione

Questi Data Transfer Object (DTO) rappresentano le strutture dati arricchite con le informazioni spaziali. A differenza del NodeDTO usato dalla Sidebar, qui i nodi includono coordinate assolute per il posizionamento sulla mappa.

Attributi

- **NodeDTO:**
 - + id: String — identificativo del nodo.
 - + type: NodeType — enumerativo che distingue tra nodo decisionale (decision) e nodo finale (leaf).
 - + text: String — testo da mostrare.
 - + resultState: String | null — eventuale esito finale (se di tipo leaf).
 - + x: Float, + y: Float — coordinate cartesiane calcolate per il centro del nodo.
- **EdgeDTO:**
 - + id: String — identificativo dell'arco.
 - + sourceId: String, + targetId: String — ID dei nodi collegati.
 - + startX, startY, endX, endY: Float — coordinate esatte dei punti di inizio e fine della linea di collegamento.
- **LayoutResult:**
 - + nodes: List<NodeDTO> — lista di tutti i nodi posizionati.
 - + edges: List<EdgeDTO> — lista di tutti gli archi posizionati.

5.9.2.2) LayoutEngine e D3LayoutEngine

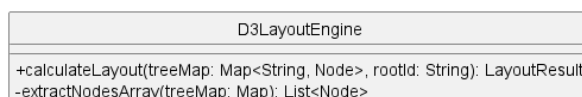


Figura 177: D3LayoutEngine

Descrizione

LayoutEngine è l'interfaccia che definisce il contratto per il calcolo spaziale. *D3LayoutEngine* ne è l'implementazione concreta, che sfrutta la libreria matematica